A dual-track technical reference

August 2026

Vendor-neutral community pattern | Apache 2.0 | All measurements from live SAP BDC tenants

This paper documents a dual-track pattern for incremental capture from SAP Business Data Cloud catalog-linked shares in Snowflake. The standard CDC mechanisms are refused; two alternative signals exist. Both tracks share one engine: dynamic tables with REFRESH_MODE = INCREMENTAL. A headline risk - silent metadata corruption in local copies - applies to both. All claims carry their measured evidence.

1. Overview

SAP Business Data Cloud (BDC) makes data products available in Snowflake as catalog-linked databases - Iceberg tables whose metadata is managed by SAP, not by Snowflake. Snowflake can read them, but the three standard change-data-capture mechanisms are all refused. Two alternative signals exist; which one applies depends on the kind of data product. Both tracks share the same engine: dynamic tables with REFRESH_MODE = INCREMENTAL.

This document records the mechanism, the measured evidence, and a risk that affects both tracks: copying the share breaks its metadata silently and without any error. The recommended first step on any new tenant is the readiness probe in probes/cdc_readiness.py. It classifies every object by track, validates the standard metadata, measures poll and extraction cost, and returns a verdict in under a minute.

2. Why standard CDC is unavailable

A catalog-linked database reports kind = TABLE and is_external = N, which suggests ordinary tables and ordinary CDC options. The objects are Iceberg tables on an external SAP-managed catalog, and Snowflake's own error messages are the clearest evidence of the real situation:

Attempted operation	Error returned
CREATE STREAM ... ON TABLE	Streams on External Tables or Iceberg tables with an external catalog must have INSERT_ONLY set to true.
SELECT ... CHANGES(INFORMATION => DEFAULT)	CHANGES clause is not supported on external tables or Iceberg tables with an external catalog.
ALTER TABLE ... SET CHANGE_TRACKING = TRUE	change_tracking cannot be set for Iceberg tables unless they use Snowflake catalog.

SHOW TABLES reports change_tracking = ON on catalog-linked objects. Streams and CHANGES() are refused regardless. That ON is misleading and should not be taken as evidence that the mechanisms will work.

INSERT_ONLY streams are permitted and do not error on creation. They see inserts and updates but cannot detect physical deletes, and the 14-day staleness window applies. There is no situation in which the INSERT_ONLY path is preferable to a dynamic table.

3. Two tracks, one engine

Which change signal is available depends on the kind of data product. Custom data products expose a source-system timestamp configured by the pipeline owner. Standard data products expose four connector metadata columns generated by SAP with no source-system configuration required. The capture engine is the same on both tracks: dynamic tables with REFRESH_MODE pinned to INCREMENTAL.

	Track 1 - Custom products	Track 2 - Standard products
Change signal	Source-system timestamp (e.g. ZZLASTCHGDATE), added by SLT pipeline configuration	__OPERATION_TYPE and __TIMESTAMP, generated by the connector

	Track 1 - Custom products	Track 2 - Standard products
Requires source configuration	Yes	No
Operation column	None - deletes are invisible on this track	Present; values observed: L=initial, A=delta. Delete code: unverified (see section 5)
Watermark comparison	Parse as TIMESTAMP_NTZ	Compare raw text - lexicographic equals chronological. Parsing a column expression turns a free poll into a scan.
Delete detection path	Dynamic table snapshot diff only	Dynamic table snapshot diff; operation column available but delete code unverified

4. Track 1 - Custom data products

The watermark column

The replication timestamp is a customer extension field added to the SAP source during SLT pipeline configuration. On the reference tenant it was named ZZLASTCHGDATE, typed TIMESTAMP_NTZ(6), and present on all 23 objects in the share. Across the full reference account: 59 objects carry it. The name varies by pipeline implementation; do not assume a name. The readiness probe discovers it.

Finding it requires reading the column list with DESCRIBE TABLE and looking for a timestamp column whose name suggests a replication event rather than a business event. A business date such as PostingDate or DocumentDate is not a replication watermark. The key distinction: does the value change when the connector writes the row, or when the business event occurred?

Why the watermark prunes

The BDC connector merges rather than reloads. Historical rows keep their original replication timestamp; new data lands in new Parquet files. Because the connector writes timestamps in monotonic, non-overlapping batches, per-file Iceberg statistics span a narrow range. Snowflake's query planner reads those statistics and eliminates every file whose range cannot contain a match.

This is a measured property of the connector, not a documented guarantee. Verify it on your tenant using the readiness probe before deploying. The merge-versus-reload check in the probe is the most critical: if the connector reloads, the watermark path reads every row every cycle and dynamic tables rebuild fully on every refresh.

Measured scan cost (reference tenant, USE_CACHED_RESULT = FALSE)

Query	Data scanned	Duration
Full column read, 108,615,680-row GL line-item object	7,035.80 MB	-
Watermark predicate, 30 matching rows	0.00 MB	28 ms
Business-date predicate, 549 matching rows	4,695.75 MB	542 ms
Full GROUP BY aggregate, 108,615,638 rows	6.87 GB	15,078 ms
Same aggregate scoped by watermark	0 GB	630 ms

Query	Data scanned	Duration
Polling the current watermark value	0.00 MB	47-79 ms

The business-date predicate scanned 4.7 GB for the same answer as the watermark predicate because business dates are scattered across every file - per-file statistics span the full historical range. The difference is not a performance optimisation: it is the difference between reading only changed data and scanning the full object.

Dynamic table results (reference tenant)

10 of 10 Gold-layer dynamic tables were confirmed INCREMENTAL once REFRESH_MODE was pinned explicitly. Initial refresh inserted 4,314,292 rows in 18.3 s. The subsequent no-change refresh returned NO_DATA in 984 ms - Snowflake established that nothing had changed without reading the source data.

Delete semantics on Track 1

There is no operation column on the custom track. A row that disappears from the source leaves no trace in the data; the watermark window will never encounter it. The only detection is a count comparison between source and target. A persistent positive divergence means rows were deleted upstream. Reconcile row counts on a schedule; rebuilding the target is the recommended response to a confirmed divergence.

5. Track 2 - Standard data products

The connector metadata columns

SAP generates four metadata columns on standard data products. Nothing needs to be configured in the source system.

Column name	Description	Type
__OPERATION_TYPE	Operation code. Values observed: L=initial load, A=delta append. Delete code: not observed and not confirmed by SAP documentation (unverified - see note below).	TEXT
__TIMESTAMP	Replication timestamp. Format: ISO-8601 with a COMMA decimal separator (2026-06-15T09:41:59,6426580Z). Arrives as TEXT.	TEXT
load_type_<32-hex-hash>	Snapshot or delta marker. Hash identifies the replication flow, not the object. Identical across all 7 objects of the reference product.	TEXT
run_id_<32-hex-hash>	Flow identifier. Held the SAME value on both the initial snapshot and the delta rows. Not a batch discriminator; cannot serve as a watermark.	TEXT

** The authoritative enumeration of __OPERATION_TYPE could not be confirmed: the SAP Help page is JavaScript-rendered. The readiness probe discovers the value distribution per tenant rather than assuming one. Do not assume a delete code exists until one is observed on your tenant.*

The __TIMESTAMP format trap

On the reference account __TIMESTAMP arrives as: 2026-06-15T09:41:59,6426580Z (a comma as the decimal separator, not a period). TRY_TO_TIMESTAMP_NTZ does not error on this; it returns NULL for

every row. An unguarded cast looks like an empty column rather than a format mismatch. Parse with: TRY_TO_TIMESTAMP_NTZ(REPLACE(__TIMESTAMP, ',', '.')).

Compare as text; parse only for display

Because the format is fixed-width and zero-padded, lexicographic order equals chronological order. This was verified directly on the reference data: parse(MAX(raw)) == MAX(parse(raw)).

Operation	Data scanned	Latency	How
MAX(__TIMESTAMP) as raw TEXT	0.000 MB	2 ms	Answered from Iceberg manifest statistics
MAX(TRY_TO_TIMESTAMP_NTZ(REPLACE(__TIMESTAMP, ',', '.')))	0.310 MB	45 ms	Must scan micropartitions

Using a parsed expression for polling turns a free manifest-statistics lookup into a micropartition scan on every cycle. Compare as text; parse in application code for display only.

Operation distribution (reference product, 5 non-empty objects)

Code	Row count	Meaning
L	67,087	Initial load snapshot
A	393	Delta (append / update)
(delete code)	0 observed	Unverified - no delete code seen on reference product

End-to-end standard-track engine (reference account)

A dynamic table built over the live share at REFRESH_MODE = INCREMENTAL covered 3,355 rows. Poll 1 returned: REFRESHED to watermark 2026-08-19T04:32:03,9099180Z. Poll 2 returned: NO CHANGE. Snowflake refresh history: INCREMENTAL in 5,241 ms, then NO_DATA in 610 ms.

Extraction cost on Track 2

Predicate pruning was NOT demonstrated at reference scale. The reference objects were a single micropartition (39,462 rows, 320 KB) and an incremental cycle scanned approximately 100% of a full read. Conclusion: drive refreshes from the free poll, but let dynamic tables do the recomputation rather than hand-written incremental MERGEs. Do not claim predicate pruning on the standard track.

6. The headline risk: never trust a local copy

> The connector metadata columns were reliable only inside the live catalog-linked database. Every local copy of a standard product on the reference account had corrupted metadata. None of this fails loudly - it produces a wrong incremental result quietly and propagates downstream.

Three separate local copies were examined. All three were corrupted:

Copy	Defect observed	Affected rows
Copy 1	__OPERATION_TYPE held media titles ('Mission Possible 1', 'About a Girl - Series') and 16,317 NULLs. 17 distinct values, up to 33 characters.	-
Copy 2	__OPERATION_TYPE, __TIMESTAMP, and LOAD_TYPE all held customer identifiers (620 distinct values) on every row. RUN_ID was NULL. Confirmed with positional SELECT * - it is the data, not a query artefact.	735,305 of 735,305
Copy 3	Same degeneracy as Copy 2.	557,693 of 558,011

The defect propagated downstream. Medallion layers built on those copies inherited the corrupted column values. A downstream consumer filtering on __OPERATION_TYPE = 'A' against a corrupted copy would return zero rows or rows matched on random customer identifiers, with no error raised.

Local copies also lose the 32-hex hash suffix on the metadata column names (load_type_, run_id_). The hash was identical across all 7 objects of the reference product. Its presence is a useful marker of a genuine live share; its absence is a signal that something is wrong.

Design rules that follow

Rule	Rationale
Point capture at the catalog-linked database, never at a copy.	Metadata is reliable only in the live share.
Validate the metadata before building on it.	Column name alone is not sufficient evidence. Check value distribution.
Treat the hash suffix as a liveness marker.	Copies lose it. Its presence confirms a live share.
Reconcile row counts on a schedule, regardless of track.	Deletes are unrepresented on Track 1 and unverified on Track 2.

7. The engine: dynamic tables

Dynamic tables with REFRESH_MODE = INCREMENTAL are the recommended engine on both tracks. They require no procedural code and are the only mechanism that captures physical deletes, by diffing consecutive source snapshots. The watermark path is the exception, not the default; use it only where a dynamic table cannot serve.

REFRESH_MODE must be pinned explicitly

Never leave REFRESH_MODE as AUTO. AUTO allows Snowflake to fall back silently to FULL when it cannot support incremental for a given query. A FULL refresh over hundreds of millions of rows carries a completely different cost profile, and nothing in the object's behaviour announces the difference - no error, no warning.

FULL also cascades: change tracking is unsupported on a FULL dynamic table, which forces everything downstream to FULL as well. Stating REFRESH_MODE = INCREMENTAL converts a silent cost regression into a deployment-time error, which is what you want.

Four verified query shapes (reference tenant)

Shape	Description	NO_DATA duration
A	Flat projection over a view that casts dates	348 ms
B	Join of two views	486 ms
C	GROUP BY aggregate	450 ms
D	Left join to a pre-aggregated child CTE	570 ms

All four shapes were accepted as INCREMENTAL with refresh_mode_reason = NULL. Initial refreshes were INCREMENTAL with numInsertedRows equal to the full row count of the source.

What blocks incremental refresh

Non-deterministic expressions inside a dynamic table's defining query force a full refresh on every cycle: CURRENT_DATE(), CURRENT_TIMESTAMP(), RANDOM(). Month-to-date logic is the most frequent source of this in practice. Put time-relative logic in a view over the dynamic table; the view is always current and adds no refresh cost.

Freshness measurement

Use SYSDATE() (UTC, TIMESTAMP_NTZ) not CURRENT_TIMESTAMP() (session-timezone aware) when measuring connector freshness. On the reference tenant, mixing them reported a real 109-minute lag as -311 minutes.

Mechanism comparison

	Dynamic table	Watermark + MERGE	INSERT_ONLY stream
Inserts	Yes	Yes	Yes
Updates	Yes	Yes	Yes
Physical deletes	Yes - diffs snapshots	No - no operation column	No - by definition
Cheap no-change detection	YES - NO_DATA in <1 s	Yes - 0 MB scanned	Yes
Code required	None	Task + control table + MERGE	Task + MERGE
Can feed targets outside Snowflake	No	Yes	Yes
Goes stale if unread	No	No	Yes - 14 days
Blocked by non-deterministic SQL	Yes	No	No

8. Operating concerns

Uninitialized objects

A share can advertise a table that was never materialised. One object on the reference account could not be DESCRIBEd: "table 'refresh_table' is not initialized". Tooling must tolerate per-object failure rather than abort the entire run. Ask the pipeline owner whether uninitialised objects are expected.

Column discovery

INFORMATION_SCHEMA.COLUMNS returns no rows for a catalog-linked database. DESCRIBE TABLE is the only route to a column list, one statement per object. Schema names contain a colon (e.g. "dp_finance_v2_srv:v1") and object names are mixed case; every reference in SQL must be double-quoted.

Monitoring

Deploy sql/03_monitoring.sql. The incremental pattern depends on the connector merging. If the connector ever switches to reload, dynamic tables stay correct while becoming as expensive as a full rebuild on every refresh. That transition raises no error. VW_RELOAD_DETECTION is what detects it.

9. Decision guide

Situation	Recommended path
Standard data product available; metadata validated	Dynamic table, REFRESH_MODE = INCREMENTAL, against the live catalog-linked database. Poll __TIMESTAMP as raw text.
Custom product; watermark column present; connector merges	Dynamic table, REFRESH_MODE = INCREMENTAL. Watermark path as exception for non-deterministic transforms or external targets.
No watermark column; connector merges	Dynamic table only. No watermark-based path is available.
Connector reloads (merge_or_reload check fails)	Dynamic tables and watermark both scan fully every cycle. Correct but expensive. Raise with SAP BDC pipeline owner.
Object shows NOT ASSESSABLE on probe	Resolve the access or initialisation issue before deploying.

10. Open asks to SAP

The following gaps would reduce deployment risk and improve the reliability of the pattern:

Ask	Reason
(a) Authoritative enumeration and contract for __OPERATION_TYPE, including whether a delete code is ever emitted.	No delete code was observed on the reference product. One may exist but had not been emitted. Without confirmation, delete handling cannot be implemented with confidence.
(b) Why __TIMESTAMP ships as TEXT with a comma decimal separator rather than a real timestamp type.	TRY_TO_TIMESTAMP_NTZ silently returns NULL on the raw value. A standard timestamp type would remove a hidden failure mode.
(c) Whether change_tracking = ON in SHOW TABLES is meaningful when streams and CHANGES() are refused.	The flag is misleading and causes diagnostic errors on first encounter.
(d) Declared primary keys per catalog-linked object.	No primary key is declared. Wrong merge grain silently discards real records without any constraint violation.
(e) Guidance on uninitialised objects.	A share can advertise a table that has never been materialised. Tooling must handle this per-object failure gracefully.

11. The readiness probe

The repo ships probes/cdc_readiness.py, which should be the first step on any new tenant. It classifies every object by track, validates the standard metadata columns (value distribution, not just column name), measures poll and extraction cost, checks the connector merge-versus-reload pattern, and returns a verdict.

```
cd probes
pip install -r requirements.txt
python3 cdc_readiness.py --connection --report readiness.md

# Add --with-scratch-tests to prove the mechanism directly
# (creates a dynamic table in a scratch database then drops it)
```

Verdict	Meaning and action
READY	All checks passed. Proceed with deployment.
READY WITH CAVEATS	Core mechanism works; at least one signal is degraded. Check which probe is flagged and read its output. Dynamic tables remain the recommended engine.
NOT READY	A critical check failed. Most likely: connector reloads, no watermark column found, or share is absent. Do not deploy until the cause is understood.
NOT ASSESSABLE	A probe could not run, usually insufficient privilege or no catalog-linked database found. Resolve and re-run.